

AI Implementation

Type of AI

The artificial intelligence utilised in this context is an auto-encoder. An auto-encoder is composed of two neural networks. The initial data is processed by the encoder, which reduces it to a space with dimensions smaller than the original data. This space is referred to as the latent space. This reduction of data enables the AI to discern the discriminating features of the data. The second neural network, designated as the decoder, endeavours to reconstruct data that is analogous in size to the data utilised as input by the encoder.

The auto-encoder

In the present implementation, the auto-encoder is coded in the “autoencoder.py” file and corresponds to the class of that same file. The system has been segmented into two distinct components: an encoder and a decoder, each defined within a separate function. The utilisation of dense layers is a hallmark of these neural networks. This approach enables straightforward and adaptable implementation, allowing for the adjustment of layer dimensions based on the efficacy of the learning process. The auto-encoder class is also equipped with a function that facilitates the construction of the auto-encoder from the encoder and decoder. These three functions are subsequently invoked within a function that constructs and compiles the model, thereby enabling subsequent training. The final function is responsible for saving the AI parameters to a file following the training process. This enables the utilisation of the AI without the necessity of additional retraining.

The training

With respect to the process of artificial intelligence (AI) learning, all activities are conducted within the “train_autoencoder.py” file. The function of this file is to retrieve data and process it (using the data_treatment class in the “data_treatment.py” file), then create an auto-encoder and compile it as defined in the autoencoder.py file. The process of learning is then initiated, taking into account the number of epochs defined at the beginning of the “train_autoencoder.py” file code (this number is obtained experimentally) and with the specific requirement that the data reconstructed by the auto-encoder must be identical to the input data. This training is exclusively performed on valid (or positive) data, i.e., on normal connection logs, as the objective of our auto-encoder is to detect anomalous connections (i.e., anomalous logs) by failing to reconstruct them (unlike normal logs, which it must be able to reconstruct thanks to its training). Subsequent to the training phase, the AI parameters are stored.

Data treatment

The data processing is executed in the “data_treatment.py” file. The retrieved file contains the CSV file containing the logs, which are then processed. The initial processing stage entails the elimination of all data deemed irrelevant to the differentiation between normal and anomalous logs. Subsequently, the data representing a date is transformed to convert it into DateTime, and the desired components (time and days of the week) are extracted. These components are transformed into sine and cosine functions to facilitate the comprehension of the cyclical nature of the data by

the AI. Subsequent to this, the geographical data is simply separated into latitudes and longitudes. The utilisation of a conventional OneHotEncoder proved to be inadequate for the processing of textual data. Consequently, the development of a bespoke encoder was necessary to address this limitation. The decision to code this system internally, as opposed to utilising a pre-existing solution, enables the delineation of two distinct phases. The first phase is the one in which the encoder perceives all potential values. The second phase is the one in which the encoder encodes the data. This separation is significant because, although the autoencoder is trained on normal data, it is essential for the dimensions of the encoded values to be correct. That is, the Custom OneHotEncoder must have seen all possible values. The rationale behind this design choice is to ensure the delineation of value dictionaries and the encoding of functions in distinct functions of the `data_treatment` class. The Custom OneHotEncoder is defined in the “`CustomeOneHotEncoder.py`” file, which contains a dictionary creation function and a data encoding function. After extensive analysis, it was determined that the numerical data should be encoded using `MinMaxScaler` on specific columns of the `DataFrame`. This process is essential for normalising the data and ensuring its integrity. The functions for encoding textual and numerical data require the specification of the `DataFrame` used, as it is possible that the filtering function defined in “`data_treatment.py`” was used to retain only normal data or only anomalous data.

API

The conventional utilisation of artificial intelligence (AI) following the completion of training is executed in the file known as “`ai_api.py`.” The initial step involves the reception of data, which is obtained from the CSV file. This data undergoes a processing procedure analogous to the training phase. However, to assess the efficacy of the auto-encoder, its application is extended to both positive (normal) and negative (anomalous) data sets. Consequently, during the data processing stage, the original `DataFrame` is divided into two disjoint `DataFrame`. Subsequently, the auto-encoder is called with the parameters stored in the Keras file, and the distance between the output vector and the input vector is measured. The distance is displayed, along with a label that indicates whether the log is normal or not.

Results issues

The findings are inconclusive due to the identical distance between the input and output data for both positive and negative data sets. However, given that the auto-encoder is trained on standard data, it is expected to exhibit a low distance on positive data and a high distance on negative data.